

Likewise Client Interview Expectations

.NET + Angular + AWS – Preparation Reference

1. Overall Client Expectation

The candidate should be a **hands-on Full Stack Engineer** who can explain:

- Current/recent project **architecture**
- Their actual **roles and responsibilities**
- Features/modules personally implemented
- Technical/design decisions made
- End-to-end flow from **Angular** → **API** → **Microservice** → **Database** → **AWS**
- Production issues handled
- Performance/scalability considerations
- Deployment and cloud implementation

The interviewer normally goes deeper based on whatever the candidate mentions.

Important: Do not mention a technology or pattern unless you can explain its **real project usage, implementation, reason for choosing it, and alternatives**.

2. Expected Technology Stack

Area	Expected Stack
Frontend	Angular
Backend	C#, .NET / .NET Core, Web API
ORM	Entity Framework / EF Core
Database	SQL Server / Relational DB
Architecture	Microservices, REST APIs, Event-Driven Architecture
Authentication	JWT, OAuth, SSO
Cloud	AWS

Containers	Docker, ECS; Kubernetes awareness
API Layer	AWS API Gateway
Messaging	AWS messaging/event services
Monitoring	OpenTelemetry, Grafana / Cloud monitoring concepts
DevOps	CI/CD, Docker image build/deployment, DB deployment
Security	JWT/OAuth, IAM, Cognito, WAF, API security

3. .NET / C# – Core Expectation

Must Be Strong

C# / OOP

Candidate should clearly understand:

- OOP principles
- Abstraction
- Encapsulation
- Inheritance
- Polymorphism
- Interface vs Abstract class
- Abstract vs Virtual methods
- `this` vs `base`
- Static class/object
- Sealed class
- Private constructor
- Extension methods
- `var` vs `dynamic`
- Record types

Questions are often asked with **practical scenarios**, not definitions alone.

SOLID Principles

Very frequently asked.

Be prepared for:

- What is SOLID?
- Why do we use SOLID?
- Real implementation from your project
- Open/Closed Principle
- Liskov Substitution Principle
- Dependency Inversion

Expected Answer Style

Don't say only:

Open for extension and closed for modification.

Explain:

- What problem existed
 - Which class/design was changed
 - How interfaces/polymorphism were used
 - How future changes can be added without modifying existing working code
-

4. Dependency Injection

This is one of the **highest-priority Likewize topics**.

Candidate should know:

DI Fundamentals

- What is Dependency Injection?
- Why DI?
- How DI reduces coupling
- Constructor injection
- IOC containers
- Built-in .NET DI
- Third-party IOC containers

DI Lifetimes

Must clearly understand:

- **Singleton**
- **Scoped**
- **Transient**

Expected scenario questions:

- Which lifetime would you choose for a DB/data service?
- Why Scoped instead of Singleton?
- When would Transient be appropriate?
- What happens if the wrong lifetime is selected?

Candidates should give **real application examples** for all three.

5. Design Patterns

Frequently expected:

Essential

- Repository Pattern
- Unit of Work
- Singleton
- Dependency Injection
- CQRS

For Senior Candidates / Microservices

- Saga
- CQRS
- Event-driven patterns

Candidate should be able to answer:

- Why was the pattern needed?
 - What problem did it solve?
 - Where exactly did you implement it?
 - What would happen without it?
-

6. Entity Framework / Data Access

Must prepare:

- Entity Framework / EF Core
- ORM concept
- Different ORM approaches
- Code First vs Database First
- EF migrations
- Fluent API
- Relationships
 - One-to-One
 - One-to-Many
 - Many-to-Many
- Calling Stored Procedures using EF
- Transactions
- Commit / Rollback
- Repository + Unit of Work with EF

LINQ

Important questions:

- `IEnumerable` vs `IQueryable`
- LINQ joins
- Left outer join
- Filtering
- Top N queries
- Query execution behavior

Candidate should understand what happens at the **database vs application-memory level**.

7. Async Programming

Frequently asked in recent discussions.

Prepare:

- `async / await`
- Task vs Thread

- Thread Pool
- Sequential dependency between async methods
- Parallel execution
- What happens to the calling thread during `await`
- End-to-end asynchronous execution

Expected scenario:

Angular

→ API

→ Service

→ Database

→ Response

Candidate should explain where asynchronous execution helps and where it does not.

8. Authentication & Authorization

This is another **very high-priority area**.

Backend

Must understand:

- JWT
- OAuth
- SSO
- Access Token
- Refresh Token
- Token generation
- Token validation
- Token expiration
- Claims
- Roles
- Permissions
- Token tampering detection
- Securing API endpoints

Know the difference between:

- Authentication

- Authorization
- Role-based authorization
- Claim-based authorization
- Permission-based authorization

SSO / Identity

Awareness expected around:

- OAuth
 - Identity Provider
 - IdentityServer concepts
 - AWS Cognito
 - AWS IAM
-

9. Angular – Core Expectation

Likewise expects **real Angular hands-on experience**, not only basic awareness.

Candidate should be able to explain a complete Angular feature they personally implemented.

Components

Prepare:

- Components
 - Component lifecycle
 - Parent → Child communication
 - Child → Parent communication
 - Communication between unrelated components
 - `@Input`
 - `@Output`
 - Services
-

10. Angular Authentication

Extremely frequently asked.

Candidate should clearly explain:

Login Flow

User Login

- Backend authentication
- Access/Refresh token
- Frontend token handling
- Interceptor
- API Request
- Token validation
- Authorization

Prepare:

- Where should tokens be stored?
 - LocalStorage vs SessionStorage
 - Security concerns
 - Token expiration
 - Refresh token flow
 - Secure token handling
 - What happens when access token expires?
 - How does every API request receive the token?
-

11. Angular Interceptors

Must know:

- Purpose of HTTP Interceptor
- Attaching authentication tokens
- Centralized request handling
- Response handling
- Error handling
- Retry failed requests
- Token refresh

Candidate should preferably explain the actual interceptor used in their project.

12. Angular Routing & Authorization

Prepare:

- Angular Routing
- RouterOutlet
- Route Guards
- Role-based page access

Common scenario:

Admin can access User Management, but regular users cannot.

Explain both:

Frontend restriction using Angular Guard

AND

Backend authorization

Frontend security alone is not sufficient.

13. Angular RxJS

Prepare strongly:

- Observable
- Promise vs Observable
- Subject
- BehaviorSubject
- Common RxJS operators
- API response handling
- State/data sharing

For experienced Angular candidates, also prepare:

- State Management
- Reducers
- View Encapsulation
 - Emulated
 - Shadow DOM

- None
-

14. Angular Configuration & Deployment

Prepare:

- DEV / QA / Stage / Production configurations
 - Environment-specific configuration
 - Angular build
 - Manual deployment
 - Automated deployment / CI-CD
 - API endpoint/environment management
-

15. SQL – Core Expectations

SQL is given significant importance.

Candidate should be comfortable with:

SQL Fundamentals

- Joins
 - Subqueries
 - CTE
 - Temp tables
 - Table variables
 - Stored Procedures
 - Functions
 - Indexes
 - Views
 - Transactions
-

16. SQL Performance

Very important.

Prepare practical troubleshooting for:

"Production stored procedure/query is slow. What will you do?"

Candidate should discuss:

- Execution plan
 - Index usage
 - Table scans
 - Joins
 - Query structure
 - Statistics
 - Large datasets
 - Blocking
 - Deadlocks
 - Parameter issues
 - Query optimization
 - Monitoring before/after changes
-

17. SQL Indexes

Prepare:

- Clustered Index
 - Non-clustered Index
 - Primary Key
 - Clustered vs Primary Key
 - Advantages/disadvantages
 - When indexes help
 - When indexes hurt
 - Indexing string/VARCHAR fields
-

18. Transactions & Isolation

Frequently asked.

Must know:

- Transactions
- Commit
- Rollback
- Isolation levels
- Concurrency
- Deadlocks
- Row versioning

For microservices, also understand why a normal DB transaction cannot simply span multiple independently deployed services.

19. Database Production Management

Recent interviews expect:

- Database deployment
- Rollback DB scripts
- Migration handling
- Production data changes
- Primary / Secondary
- Master / Slave
- Read replicas
- Database-per-microservice considerations

Candidates should explain how schema changes are safely promoted through environments.

20. Microservices – High Priority

This has become a **major client expectation**.

Candidates should explain their actual architecture.

Prepare:

- Why Microservices?
- Number of services
- Responsibility of each service

- Service boundaries
- Database strategy
- API Gateway
- Authentication
- Service-to-service communication
- Error handling
- Deployment
- Monitoring
- Scaling

Avoid saying simply:

"We used 10 microservices."

Be able to explain at least **2–3 important services end-to-end**.

21. Service-to-Service Communication

Must understand:

Synchronous

- REST / HTTP

Asynchronous

- Events
- Queues
- Messaging

Be able to explain:

- When REST is appropriate
 - When event-driven communication is better
 - Retry
 - Failure handling
 - Loose coupling
 - Payload considerations
-

22. Event-Driven Architecture

Recent Likewize interviews specifically test:

- Why event-driven architecture?
- When should it be used?
- REST vs Events
- Large event payload handling
- Async processing

Example expected scenario:

Instead of putting a large file/message directly into an event:

Application

- Store large object externally
- Publish reference/location in event
- Consumer retrieves object

Candidate should understand the principle rather than memorize one AWS service.

23. CQRS

Do not prepare only the definition.

Explain:

- Command / Write
- Query / Read
- Why separate them
- When separation provides value
- Different read/write models
- Scalability scenario
- When CQRS would be unnecessary

Likewize interviewers often ask:

"Why did you use CQRS? Give me the exact scenario."

24. Saga Pattern

For senior/microservice candidates:

Understand distributed transactions such as:

Order

→ Payment

→ Inventory

→ Shipment

If a later operation fails, understand:

- Compensating action
 - Event orchestration/choreography concepts
 - Why normal SQL rollback cannot cover independent microservices
-

25. AWS – Major Client Expectation

Recent feedback clearly indicates candidates need **practical AWS knowledge**.

Basic theoretical AWS knowledge may not be enough.

Candidate should know the AWS services actually used in the project and be able to explain:

Why was this service selected?

26. AWS API Gateway

Prepare:

- Why API Gateway?
- Request routing
- Authentication
- Authorization
- Integration with backend services
- Rate limiting/throttling concepts
- Microservice exposure

- Security

Candidate should connect API Gateway with the architecture instead of describing it independently.

27. AWS Authentication & Security

Prepare:

IAM

- Users/Roles/Policies
- Least privilege
- Service permissions

Cognito

- User authentication
- Tokens
- Application integration

WAF

- Why WAF?
- Protecting public APIs/web applications

Candidates should explain security at multiple layers.

28. AWS Containers – Very Important

Recent interview questions expect strong practical understanding.

Prepare the complete flow:

Source Code

→ CI Pipeline

→ Docker Build

→ Docker Image

- Container Registry
- ECS
- Task Definition
- Service
- Load Balancer/API Gateway
- Running Container

Be prepared for:

- How do you create the Docker image?
 - Where is the Docker image stored?
 - How is ECS started?
 - How do containers receive configuration?
 - How does the application scale?
 - How are new versions deployed?
 - How do you troubleshoot failed containers?
-

29. Docker

Must know:

- Dockerfile
- Image
- Container
- Build
- Tag
- Registry
- Environment variables
- Ports
- Logs
- Deployment

Candidates claiming AWS microservice experience should be comfortable describing Docker practically.

30. Kubernetes

Not necessarily mandatory for every position, but recent interviews check exposure.

Prepare basic understanding of:

- Pods
- Deployments
- Services
- Scaling
- Container orchestration
- Configuration/secrets

Do not claim deep Kubernetes experience unless you have actually worked with it.

31. AWS Scalability

Important scenario:

"Application traffic suddenly increases. How will you scale it?"

Candidate should explain the actual architecture:

Traffic

- Load Balancer/API Gateway
- Multiple application/container instances
- Auto Scaling
- Database/cache/messaging considerations

Also explain:

- Horizontal scaling
 - Stateless applications
 - Bottleneck identification
 - Database scaling
-

32. AWS Cost Optimization

Recent client scenario:

Legacy application has variable traffic and a high cloud bill. How will you optimize it?

Think through:

1. Understand workload
2. Analyze utilization
3. Identify idle/oversized resources
4. Right-size compute
5. Introduce auto scaling
6. Use appropriate serverless/container model where suitable
7. Optimize storage/database
8. Review network/logging costs
9. Monitor continuously

Client expects an **engineering approach**, not simply "use Lambda."

33. Lambda / Serverless

Prepare:

- When Lambda is appropriate
- Event-driven processing
- Short-running workloads
- Scaling

Also know when **Lambda is NOT appropriate**, such as very long-running operations.

34. Monitoring & Observability

Recent interviews include:

- Application monitoring
- Exceptions
- Performance
- OpenTelemetry
- Grafana

Candidate should understand:

Application

→ Logs + Metrics + Traces

- Monitoring Dashboard
- Alert
- Investigation
- Root Cause

Be prepared to explain tools actually used in your project.

35. Error & Exception Handling

Candidate should explain:

Backend

- Global exception handling
- Middleware
- Logging
- Standard error response
- Correlation/trace information

Angular

- Interceptor/global error handling
- User-friendly messages
- Retry where appropriate

Database

- Stored procedure error handling
 - Transactions
 - Rollback
-

36. .NET Middleware

Frequently asked.

Candidate should explain the full pipeline:

HTTP Request

- Authentication Middleware
- Authorization
- Logging / Custom Middleware
- Controller/API
- Service
- DB
- Response passes back through middleware

Interviewers may ask:

"After the response is returned, does middleware execute again?"

Understand the request/response pipeline clearly.

37. API Engineering

Prepare:

- REST API design
- Routing
- Request parameters
- Headers
- Request body
- Form data
- API versioning
- Validation
- Authentication
- Authorization
- Exception handling
- Performance

For senior engineers:

- Idempotency
 - Retry
 - Timeout
 - Large payloads
 - API Gateway
 - Service communication
-

38. Large Payload / File Handling

Recent client questions specifically test this.

Candidate should explain:

- Why very large files should not unnecessarily flow through every microservice
 - Streaming
 - Chunking
 - Object storage
 - Passing metadata/reference instead of the whole object
 - Async processing
 - Payload limits
-

39. CI/CD & Deployment

Candidate should understand the real deployment flow.

Example:

Developer

- Git
- Pull Request
- Build
- Unit Test
- Docker Image
- Registry
- Environment Deployment
- Smoke Test
- Production

Prepare:

- CI/CD
- YAML/pipeline basics
- Environment variables
- Rollback
- Configuration management
- DB deployment
- Docker deployment

40. Architecture Discussion – Critical

Before the interview, candidate must prepare a **5–7 minute architecture explanation**.

Recommended sequence:

1. Business Problem

What does the application do?

2. Users

Who uses the application?

3. Frontend

Angular architecture.

4. Authentication

JWT / OAuth / SSO.

5. API Layer

API Gateway / .NET APIs.

6. Backend

Microservices and responsibilities.

7. Communication

REST vs Events.

8. Database

DB type and ownership.

9. Cloud

AWS services.

10. Deployment

Docker / ECS / CI-CD.

11. Monitoring

Logs, metrics, tracing.

12. Candidate Contribution

Exactly what the candidate designed/developed.

41. Architecture Decision Questions

Senior candidates should prepare at least **3 real architecture decisions**.

For each:

Problem → Options → Decision → Why → Trade-off → Result

Examples:

- REST vs Event-driven
 - Monolith vs Microservices
 - ECS vs Lambda
 - SQL vs NoSQL
 - JWT/OAuth authentication
 - Repository/CQRS
 - Caching strategy
-

42. Production & Troubleshooting Scenarios

Expect scenario-based discussion.

Examples:

API is slow

Explain:

API

- Logs/Tracing
- External dependency
- Service
- DB query
- CPU/Memory
- Network
- Root Cause

Production DB query is slow

Explain:

Query

- Execution Plan
- Index
- Data Volume
- Blocking
- Optimization
- Validate

Microservice is failing

Explain:

Gateway

- Service health
 - Container
 - Logs
 - Dependency
 - DB/Event
 - Retry/Fallback
-

43. Behavioral / Ownership Expectation

Recent interviews also test engineering maturity.

Prepare scenarios around:

- Unable to complete work within deadline
- Multiple projects with unclear priorities
- Code review disagreement
- Communicating risks to manager
- Using AI coding assistants responsibly
- Handling urgent production work

Expected behavior:

Understand → Prioritize → Communicate early → Propose options → Agree on plan → Deliver → Follow up

44. Project Migration / Modernization

Recent interviews include migration questions.

Prepare:

- Monolith → Microservices
 - Legacy → Modern .NET
 - Version upgrades
 - Breaking changes
 - Coding standards
 - Sunset strategy
 - Incremental migration
 - Regression testing
 - Deployment strategy
-

45. Highest Priority Preparation Areas

Priority 1 – Must Be Very Strong

1. **Current Project Architecture**
2. **Roles & Responsibilities**
3. **.NET / C# fundamentals**
4. **SOLID + Design Patterns**
5. **Dependency Injection**

6. **JWT / OAuth / Authentication**
7. **Angular Authentication + Interceptor**
8. **Angular Components / Services / RxJS**
9. **SQL + Query Performance**
10. **Microservices Architecture**

Priority 2 – Increasingly Critical

11. **AWS Services – Practical Usage**
12. **API Gateway**
13. **Docker + ECS deployment**
14. **Service-to-Service Communication**
15. **Event-Driven Architecture**
16. **CQRS**
17. **Saga**
18. **Application Scaling**
19. **Cloud Cost Optimization**
20. **Monitoring / Observability**

Priority 3 – Supporting Knowledge

21. **CI/CD**
22. **Kubernetes fundamentals**
23. **Large payload handling**
24. **Database replicas**
25. **DB deployment/rollback**
26. **Unit testing / Mocking**
27. **Migration/modernization**

46. Client Interview Preparation Rule

For every technology mentioned in the resume, prepare these **5 questions**:

1. What is it?

Basic understanding.

2. Where did you use it?

Actual project.

3. Why did you use it?

Reason/design decision.

4. How did you implement it?

Hands-on details.

5. What problem did it solve?

Business/technical impact.

If the candidate cannot answer all five, they should strengthen that area before the Likewize discussion.

47. Recommended End-to-End Preparation Flow

Every candidate should be capable of explaining something like:

Angular

↓ Authentication / JWT / Interceptor

AWS API Gateway

↓

.NET Microservice

↓ DI / Middleware / Repository / EF Core

SQL Database

and/or

Microservice

↓ Publish Event

AWS Messaging/Event Service

↓

Another Microservice

↓

Database

Deployment:

Git → CI/CD → Docker → Registry → ECS → AWS

Monitoring:

Application → Logs/Metrics/Traces → Monitoring Dashboard

This complete flow is more important than memorizing individual definitions.

48. Final Likewize Readiness Expectation

A strong Likewize candidate should be able to say:

I can explain the application I worked on end-to-end, including Angular frontend, authentication and authorization, .NET APIs, microservices, database design, AWS infrastructure, container deployment, service communication, monitoring, scaling and the design decisions I personally contributed to.

The interview is mainly looking for **practical engineering depth, hands-on implementation and ownership**, rather than theoretical definitions.